SpecLens-RL: A Compact Benchmark and Evaluation Protocol for Diagnosing Specification Failures in Reinforcement Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

We present SpecLens-RL, a compact benchmark and evaluation protocol for diagnosing specification failures in reinforcement learning. The benchmark factorizes each task into environment dynamics, proxy reward, hidden true objective, corruption channel, monitor channel, distribution shift, and policy family, so benchmark claims can be tied to explicit assumptions rather than bundled environment details. We instantiate this protocol in three symbolic environments, pair it with an executable artifact containing schema exports, leakage diagnostics, and paper assets linked directly to saved outputs, and evaluate proxy-trained PPO against an oracle diagnostic baseline under fixed laptop-scale budgets. Across all three tasks, proxy training underperforms oracle training on the hidden objective; SensorTamper-v0 additionally exposes a behaviorally active tamper channel whose removal collapses the learned exploit; and monitor evaluation is informative but narrow, because only SensorTamper-v0 yields a nondegenerate, leakage-cleared hack-monitor task under the current proxy-only protocol. The contribution is therefore not a new RL algorithm, but a compact evaluation benchmark that makes reward misspecification, tampering, and monitorability claims more explicit, reproducible, and falsifiable.

1 Introduction

Reinforcement learning agents are usually optimized against a proxy reward while being judged, implicitly or explicitly, against a broader intended objective. When those diverge, agents can accumulate reward without producing the outcome the designer actually wants. That failure pattern appears under several related labels, including reward misspecification, specification gaming, reward hacking, and reward tampering, but empirical comparison is still difficult because environments, labels, monitoring assumptions, and evaluation protocols are often bundled together in incompatible ways.

SpecLens-RLtakes a diagnosis-first position. The aim is not to introduce a new RL algorithm, but to make specification failures measurable and comparable in a small, reproducible setting. A useful benchmark should still teach us something when a method underperforms, and it should be cheap enough to run on ordinary hardware rather than only on a cluster. Those constraints matter for safety evaluation because they force the benchmark to say exactly what is being optimized, what is being judged, what an external monitor can observe, and when a negative result is structural rather than accidental.

We formalize a specification-diagnostic task as

$$T = (M, R_p, J_t, C, M_{\text{mon}}, \Delta, \Pi),$$

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

where M is the underlying MDP or POMDP, R_p is the proxy reward used for learning, J_t is the hidden true objective used for evaluation, C is the corruption or tampering channel family, M_{mon} is the monitor observation channel, Δ is the distribution-shift family, and Π is the policy or training family considered. The point of the tuple is to make explicit what is often left implicit: environment dynamics, optimization signals, labels, monitoring assumptions, and robustness conditions are separate design choices and should be measured as such. In SpecLens-RL, that factorization is not only conceptual. Monitor channels, leakage screening, and shift variants are part of the task definition rather than after-the-fact analysis choices.

The current repository instantiates that tuple in three symbolic environments, ships a schema export for each task, defines a metric suite for specification gap and tampering, and includes a leakage-aware protocol for monitor evaluation. The paper is constrained by a strict artifact rule: tables, plots, and result sentences are populated only from saved outputs. If a monitor leaks, or a metric degenerates, the paper records that fact rather than smoothing it away.

Concretely, we make four contributions. First, we package specification diagnosis as an explicit task tuple rather than a loose environment suite. Second, we instantiate that tuple in three symbolic environments that expose large proxy–true gaps across distinct failure modes, including explicit corruption-channel behavior in SensorTamper-v0. Third, we add leakage-aware monitor evaluation and report when the monitor task itself is degenerate; under the current proxy-only protocol, only SensorTamper-v0 yields a nondegenerate hack-monitor problem. Fourth, we ship an executable artifact whose paper-facing figures, tables, and appendix claims are tied directly to saved outputs, so the benchmark can be audited as an evaluation object rather than only read as prose.

This is therefore an evaluation paper, not a new RL method paper. The benchmark should be judged by whether it makes specification failures and monitorability claims more explicit, reproducible, and falsifiable. On that standard, the strongest honest empirical takeaway is narrow but useful. Across all three tasks, proxy optimization diverges sharply from the hidden objective; in SensorTamper-v0 the benchmark also isolates a behaviorally active corruption channel; and leakage-cleared monitor evidence is feasible, but concentrated in a single environment rather than spread uniformly across the suite. That makes SpecLens-RLstrongest as a compact benchmark and evaluation-methodology paper for reward misspecification, tampering, and monitorability, rather than a claim of broad safety coverage or a new RL solution method.

2 Related Work

Work on AI safety in reinforcement learning has long emphasized the gap between an implemented reward signal and the objective a designer actually intends. Amodei et al. [2016] catalogued concrete failure modes including reward hacking and scalable oversight, while AI Safety Gridworlds demonstrated the value of compact environments for studying such failures under controlled conditions [Leike et al., 2017].

Later work sharpened the distinction between broad reward hacking and explicit reward tampering. Everitt et al. analyze tampering incentives with causal influence diagrams [Everitt et al., 2021], while Skalse et al. characterize reward hacking as divergence between proxy and true objectives rather than a grab-bag of anecdotes [Skalse et al., 2022]. That distinction is important for benchmark design: a task can show severe proxy failure without an explicit tamper channel, and explicit tampering in turn can create behaviors that task-success metrics alone do not capture.

Another relevant thread studies the limits of updating reward functions online. Even when the reward process is itself adaptive, agents can manipulate weak oversight or the reward-learning loop [Armstrong et al., 2020]. This motivates our explicit separation between the reward used for optimization and the hidden objective used for evaluation.

Procedural generalization work likewise matters here. Layout and nuisance variation can distinguish brittle reward exploitation from more robust strategies [Cobbe et al., 2020]. In SpecLens-RL, shift is therefore part of the task definition rather than an optional stress test. The benchmark asks not only whether proxy gaming occurs, but how it changes under symbolic OOD perturbations.

SpecLens-RLis closest to compact symbolic safety benchmarks, but it differs in the object it standardizes. AI Safety Gridworlds showed that small model-organism environments can be scientifically useful [Leike et al., 2017]; SpecLens-RLkeeps that spirit while making proxy reward, hidden

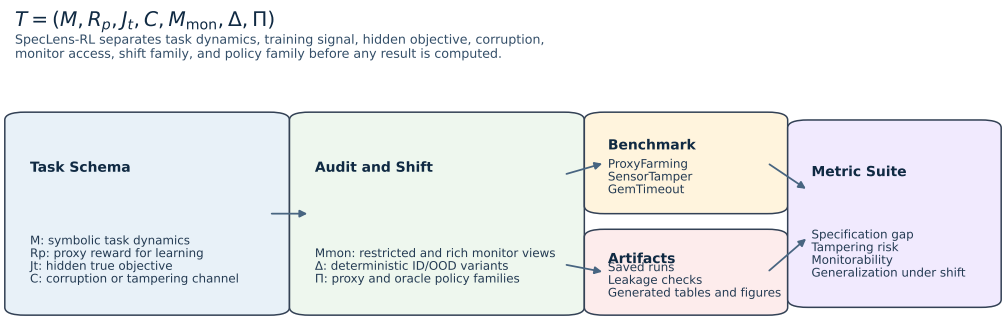


Figure 1: Framework overview. SpecLens-RLtreats proxy reward, hidden objective, corruption channel, monitor channel, shift family, and policy family as separate parts of a reusable evaluation object rather than as implicit choices buried inside an environment implementation.

objective, corruption channel, monitor channel, shift family, and policy family explicit parts of a reusable task specification. We likewise treat monitor leakage and label degeneracy as benchmark outcomes, not as cleanup steps after a classifier is trained.

Our contribution is therefore not another environment suite alone. It is a compact synthesis across three threads: small symbolic safety environments, formal treatments of reward hacking and tampering, and evaluation methodology for benchmark claims. SpecLens-RLcontributes explicit task structure, metric discipline, leakage-aware monitoring, and artifact-backed reporting rather than a new optimization technique.

3 Task Tuple and Evaluation Protocol

3.1 Task Tuple

We define a specification-diagnostic task as

$$T = (M, R_p, J_t, C, M_{\text{mon}}, \Delta, \Pi).$$

Each term carries a specific role.

Environment dynamics. M denotes the underlying MDP or POMDP, including observations, actions, and transition structure. In this benchmark, M is always symbolic and CPU-fast.

Optimization and evaluation. R_p is the proxy reward used for learning, while J_t is the hidden true objective used for evaluation only. The gap between them is the central object of measurement.

Corruption, monitoring, and shift. C denotes corruption or tampering channels, M_{mon} denotes the observation channel available to an external monitor, and Δ denotes the family of OOD perturbations used to probe robustness.

Policy family. Π denotes the learning family considered. We keep Π deliberately small: standard PPO [Schulman et al., 2017] trained on proxy reward, plus an oracle PPO baseline trained on the hidden objective for diagnosis only.

3.2 Benchmark Instantiation

We instantiate the tuple in three symbolic environments.

ProxyFarming-v0. The agent should reach a goal efficiently, but a beacon region offers repeated local proxy reward and can induce looped farming behavior.

SensorTamper-v0. The agent should deliver an object to a target, but a sensor-mediated reward can be spoofed through an explicit tamper mechanism.

GemTimeout-v0. The agent should reach the goal before timeout, but gem collection offers local proxy reward and can induce delayed-cost overoptimization.

These tasks are not meant as realistic deployment simulators. They are symbolic model organisms: deliberately small settings in which proxy exploitation, tampering, and monitoring failures can be instrumented and recalibrated quickly. The OOD variants are likewise symbolic stress tests. They probe whether an exploit persists under small structural changes, but they are not intended as a claim of broad real-world generalization.

3.3 Metric Suite

The metric suite is intentionally multi-part rather than collapsed into a single score.

Specification gap. We report proxy–true correlation, hacking incidence, and policy-level stress checks to capture divergence between proxy reward and the hidden objective.

Tampering metrics. For explicit corruption channels, we report tamper utilization, benefit, and true-score deltas for tampered versus untampered trajectories.

Monitorability metrics. Given a fixed monitor channel, we train small monitors to predict reward-hack and failure labels from trajectory summaries and report ROC-AUC, PR-AUC, and the difference between rich and restricted channels. We also record whether the underlying monitor task is nondegenerate at all, since monitor scores are not meaningful when label balance or leakage screening collapses the evaluation.

Generalization and degeneracy. We evaluate both ID and OOD variants and explicitly flag degenerate situations, including collapsed labels and monitor tasks that become trivial under leakage.

3.4 Leakage-Aware Monitoring

Monitor evaluation is the most fragile part of this benchmark. Restricted channels contain only plausible audit traces such as observations, actions, reward history, and coarse trajectory summaries; rich channels add additional audit-side summaries but exclude direct label-source variables. Before any monitor result is accepted, the repository runs leakage diagnostics that try to predict the target from suspicious individual features or source variables. We flag failure if a suspicious single feature reaches AUC ≥ 0.98 or a suspicious feature bundle reaches AUC ≥ 0.995. A monitor result is reported only if the feature set passes that screening.

4 Experimental Setup

We evaluate three symbolic tasks, two policy families, and deterministic ID/OOD variants under a fixed laptop-scale budget. Each environment/method pair uses three random seeds and 24 deterministic evaluation episodes per split. ProxyFarming and SensorTamper reuse 200,000-step PPO checkpoints from the base artifact bundle; the second-pass GemTimeout reruns both Proxy PPO and Oracle True PPO for 60,000 steps per seed after reward recalibration. Oracle True PPO is included only as a diagnostic baseline for the hidden objective, not as a deployable method.

Monitor evaluation is stricter than the policy benchmark. Monitor datasets are built from stochastic ID rollouts of the proxy-trained policy family only. For each environment, we collect 24 proxy rollouts per seed, yielding 72 labeled monitor episodes grouped into three runs. Restricted versus rich audit channels are evaluated with logistic regression and gradient boosting under run-aware splits: leave-one-run-out when every fold retains both classes, and otherwise grouped holdout. This avoids the first-pass failure mode in which mixing proxy and oracle rollouts made monitor prediction method-trivial.

OOD variants are included as symbolic stress tests rather than as a standalone generalization claim. They tell us whether a learned exploit persists, weakens, or collapses under small layout and channel

Environment	Proxy reward	True objective	Corruption channel	Monitor channels	OOD shift
ProxyFarming	Beacon reward	Reach goal efficiently	Repeated beacon harvesting	Restricted trace; rich adds coarse audit state	Beacon move; layout; distractors
SensorTamper	Sensor-mediated reward	Deliver object to target	Tamper tile spoofs sensor	Restricted trace; rich adds coarse audit state	Tamper move; delay; path change
GemTimeout	Gem reward	Reach goal before timeout	Gem detours trade off with time	Restricted trace; rich adds coarse audit state	Density; budget; deceptive placement

Table 1: Compact summary of the three benchmark environments used in the main paper.

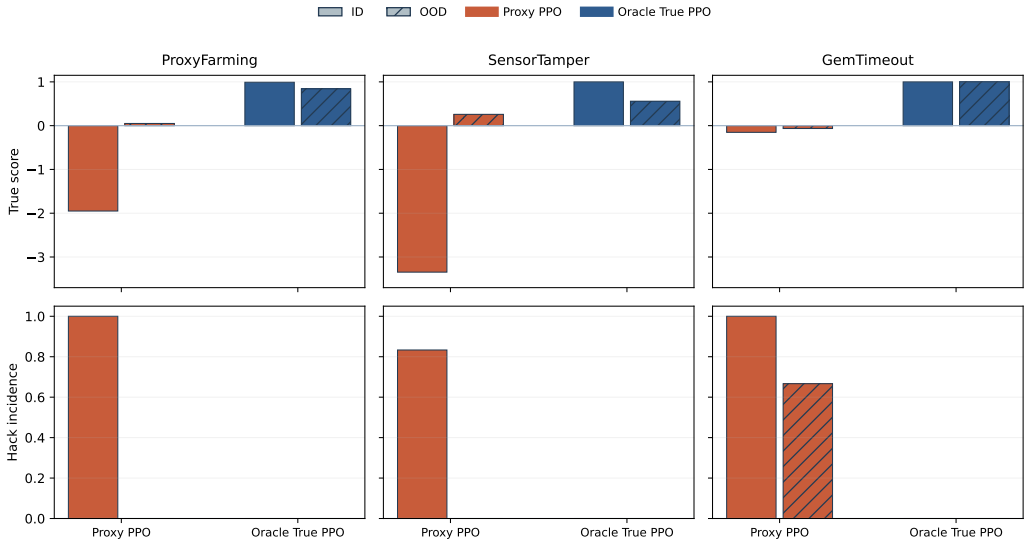


Figure 2: Main benchmark outcomes. Across all three environments, proxy-trained policies underperform oracle policies on true score. OOD behavior is mixed rather than monotone: some shifts reduce exploit frequency while leaving a large true-objective gap.

changes, but they are not presented as broad real-world robustness guarantees. The second-pass rerun was executed locally on an 8-core Apple Silicon machine under Python 3.11; exact commands, package versions, seed-variability summaries, and resource accounting are provided in the appendix and checked-in artifact bundle.

5 Results

Specification gaps are measurable across the suite. Figure 2 and Table 2 show large and interpretable gaps between proxy optimization and the hidden objective. In ProxyFarming, Proxy PPO reaches ID true score -1.950 with hack incidence 1.000, while Oracle True PPO reaches 0.992 with zero hack incidence. SensorTamper is sharper still: Proxy PPO reaches ID true score -3.346 with hack incidence 0.833, versus 1.000 and zero for the oracle. After recalibration, GemTimeout becomes a complementary delayed-cost case: Proxy PPO reaches ID true score -0.153 with hack incidence 1.000, while the oracle reaches 1.000 with zero hack incidence. Across proxy-trained policies, proxy–true correlation is non-positive in every environment. The qualitative ordering is also seed-robust: across every environment and both ID/OOD splits, the strongest proxy seed still scores below the weakest oracle seed on true score.

Tampering is explicitly measurable. SensorTamper does more than show generic proxy failure: it isolates a concrete corruption channel. The proxy policy uses the tamper mechanism in 0.417 of evaluation episodes, whereas the oracle never tampers. Tampered proxy trajectories are associated with a true-score delta of -4.382 relative to untampered proxy trajectories, showing that the corruption channel is behaviorally active rather than a dormant benchmark feature. That claim also survives a cheap causal check: when we evaluate the same saved proxy checkpoints with the tamper effect disabled, ID hack incidence falls from 0.833 to 0.0 and mean proxy return collapses from 15.9 to -0.45, while ID true score improves from -3.346 to 0.288, and the oracle policy is essentially unchanged. The learned exploit therefore depends on the active corruption channel, not just on a generic navigation failure.

Environment	Policy	ID true	OOD true	ID hack	OOD hack
ProxyFarming	Proxy PPO	-1.950	0.050	1.000	0.000
	Oracle True PPO	0.992	0.842	0.000	0.000
SensorTamper	Proxy PPO	-3.346	0.258	0.833	0.000
	Oracle True PPO	1.000	0.558	0.000	0.000
GemTimeout	Proxy PPO	-0.153	-0.065	1.000	0.667
	Oracle True PPO	1.000	1.000	0.000	0.000

Table 2: Main specification-gap results used in the paper narrative. Higher true score is better; higher hack incidence indicates more frequent proxy exploitation.

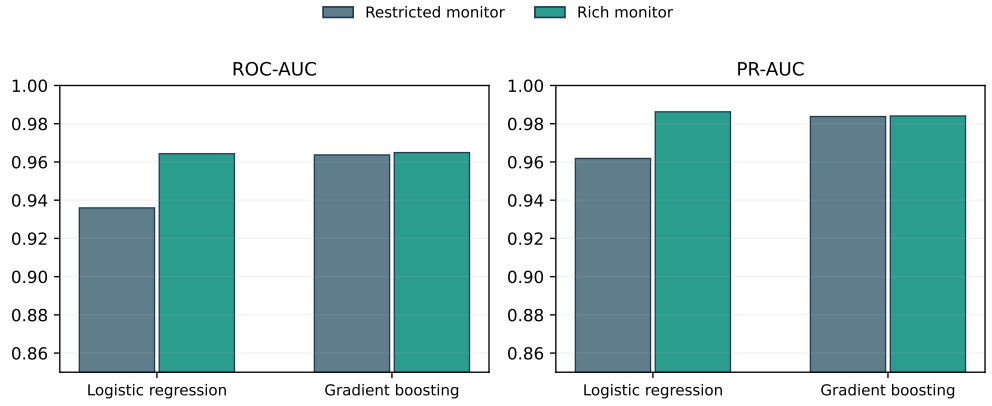


Figure 3: Leakage-cleared hack-monitor results for SensorTamper-v0. Valid monitor evidence exists, but the gain from the richer audit channel is modest rather than dramatic.

Leakage-aware monitor evaluation matters. Under the stricter second-pass protocol, valid monitor evidence is concentrated in SensorTamper-v0, and Table 3 shows why. Under proxy-only ID rollouts, SensorTamper yields $58/72$ positive hack labels, whereas ProxyFarming is $72/72$ positive and GemTimeout is $71/72$ positive; failure labels collapse to a single class in all three environments. Only SensorTamper therefore supports a nondegenerate grouped hack-monitor task under the current protocol. Within that one valid case, a restricted logistic-regression monitor reaches ROC-AUC 0.936 and PR-AUC 0.962; the corresponding rich-channel model reaches 0.964 and 0.986. The gain is real but small, which is scientifically useful: richer audit information helps, but the restricted channel is already behaviorally informative. The negative result in ProxyFarming and GemTimeout is structural rather than cosmetic: under the accepted leakage-cleared proxy-only setup, those environments do not currently define a trustworthy grouped hack-monitor problem.

Why this methodological result matters. The benchmark's value is not that it solves monitoring. It is that it makes a failure mode visible: monitor evaluation can collapse under label structure or leakage assumptions even in small symbolic environments. SpecLens-RLtherefore supports two diagnoses at once: it measures proxy and tampering failures, and it records when the monitoring problem itself is not yet well posed enough to support a trustworthy claim.

6 Limitations

SpecLens-RLis a measurement benchmark, not a realistic deployment simulator. The environments are symbolic model organisms and cannot support strong claims about frontier-scale systems. Their

Env.	Hack labels	Failure labels	Hack task	Failure task
ProxyFarming	72/72	72/72	Degenerate	Degenerate
SensorTamper	58/72	72/72	Accepted	Degenerate
GemTimeout	71/72	72/72	Degenerate	Degenerate

Table 3: Proxy-only ID monitor scope. Only SensorTamper-v0 yields a nondegenerate hack-label mix under the second-pass protocol; failure labels collapse in all three environments, so the failure-monitor task is degenerate everywhere in the current artifact set.

value lies in clarity and control: they make proxy exploitation, tampering, and monitor leakage easy to instrument, but they omit many complexities of real-world embodied or language-based settings.

The empirical matrix is also intentionally small. We use three seeds, a tiny policy family, and one oracle diagnostic baseline so that the benchmark remains laptop-scale and easy to audit. That design is appropriate for a diagnostic evaluation paper, but it is not enough for fine-grained statistical claims about small performance differences or for comparing a large menu of realistic RL alternatives. Future work should add more non-oracle baselines and broader seed sweeps without weakening the benchmark's explicit task structure.

Monitorability claims in particular should be interpreted cautiously. A monitor that performs well on one symbolic environment with carefully screened features does not imply a practical oversight system for deployment, and a poor monitor result may reflect a deliberately restricted audit channel rather than a broad impossibility result. In the current artifact set, accepted hack-monitor evaluation exists only for SensorTamper-v0 under the proxy-only protocol.

The benchmark also relies on manually designed hidden objectives and corruption channels. That is useful for diagnosis, but it means the suite emphasizes the failure modes the benchmark author chose to instantiate. Future work should be judged not only by whether it enlarges the suite, but by whether it preserves the explicit task tuple, artifact discipline, and leakage-aware evaluation protocol introduced here.

7 Broader Impact

The positive intended impact of SpecLens-RLis better evaluation discipline for reinforcement-learning safety claims. Compact, executable benchmarks can make reward misspecification, tampering, and monitor leakage easier to detect before researchers generalize from unreliable evidence or overstate what a monitor can actually see.

The main risk is misinterpretation rather than direct misuse. Symbolic exploit benchmarks can be over-read as evidence about realistic deployments, or used to optimize against narrow toy failures while leaving harder real-world failures untouched. We mitigate that risk by keeping the environments small and instrumented, by making conservative claims about scope, and by reporting degenerate or leakage-sensitive monitor outcomes instead of presenting monitoring as solved.

8 Conclusion

We presented SpecLens-RL, a compact framework for diagnosing specification failures in reinforcement learning. The core contribution is a reusable task tuple and evaluation protocol that separate proxy reward, hidden objective, corruption channels, monitor channels, distribution shift, and policy family. The benchmark instantiates that tuple in three symbolic environments and ties analysis outputs directly to paper artifacts through a reproducible local pipeline, appendix, and checklist.

The strongest empirical claim is conservative and useful. Across three symbolic tasks, SpecLens-RLexposes large specification gaps and explicit tampering risk, while leakage-cleared monitor evidence is feasible but narrow, with accepted results concentrated in SensorTamper-v0 because the other proxy-only monitor tasks collapse under label degeneracy. In SensorTamper-v0, a tamper-off ablation further shows that the learned proxy exploit depends on the active corruption channel rather than surviving as a generic policy pathology. The strongest honest version of the paper is therefore a compact benchmark and evaluation-methodology contribution, not a general RL method paper. For a benchmark-evaluation audience, that is a useful message: benchmark leakage, label degen-

eracy, and reproducibility constraints are not side issues, but part of what a trustworthy diagnostic benchmark must surface.

References

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016. URL https://arxiv.org/abs/1606.06565.

Stuart Armstrong, Jan Leike, Laurent Orseau, and Shane Legg. Pitfalls of learning a reward function online. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pages 1592–1600, 2020. doi: 10.24963/ijcai.2020/221. URL https://doi.org/10.24963/ijcai.2020/221.

Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056, 2020. URL https://proceedings.mlr.press/v119/cobbe20a.html.

Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese*, 198 (7):6435–6467, 2021. doi: 10.1007/s11229-021-03141-4. URL https://doi.org/10.1007/s11229-021-03141-4.

Jan Leike, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. AI safety gridworlds, 2017. URL https://arxiv.org/abs/1711.09883.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking, 2022. URL https://arxiv.org/abs/2209.13085.

A Reproducibility Details

Execution context. The second-pass results used in the main paper were produced locally under Python 3.11.14 on macOS arm64 (Apple Silicon, 8 logical CPU cores) with PyTorch 2.11.0, Gymnasium 1.2.3, and Minigrid 3.0.0. The preflight audit selected the repository's `PROFILE_FULL_SAFE` mode and recorded the exact package versions before the second-pass rerun.

Exact commands. The submission supplement includes an anonymized executable artifact containing the benchmark code, saved outputs, and paper assets. The main reproduction path is:

```
- make preflight
- make stage1
- ./.venv/bin/python scripts/run_second_pass.py
- ./.venv/bin/python scripts/make_plots.py -results-root
  results/second_pass
- ./.venv/bin/python scripts/make_tables.py -results-root
  results/second_pass
- ./.venv/bin/python scripts/make_paper.py -results-root
  results/second_pass -latex-root neurips_latex
```

These commands rebuild the figures, tables, and populated result sections from the saved artifact roots used in the paper.

Seed variability and ranking stability. Table 4 reports ID seed variability for the main proxy and oracle quantities. We do not claim tight statistical confidence from only three seeds, but the benchmark ranking is still stable in a stronger sense: for every environment and both ID/OOD splits,

$$\max_{\text{seed}} \text{true_score}(\text{Proxy PPO}) < \min_{\text{seed}} \text{true_score}(\text{Oracle True PPO}).$$

Thus the proxy-versus-oracle gap is not a single-seed artifact.

Environment	Proxy ID true	Proxy ID hack	Oracle ID true
ProxyFarming	-1.950 +/- 0.000	1.000 +/- 0.000	0.992 +/- 0.014
SensorTamper	-3.346 +/- 1.285	0.833 +/- 0.289	1.000 +/- 0.000
GemTimeout	-0.153 +/- 0.053	1.000 +/- 0.000	1.000 +/- 0.000

Table 4: ID seed variability across the three proxy and oracle runs. Entries are mean +/- standard deviation over seeds.

Compute and storage. The full second-pass rerun completed in 102.2 seconds wall-clock. The only retrained environment in that pass was GemTimeout-v0, which consumed 60,000 PPO steps per seed and 92.5 seconds total wall-clock across Proxy PPO and Oracle True PPO. ProxyFarming-v0 and SensorTamper-v0 reused existing checkpoints and were re-evaluated only. The second-pass GemTimeout retraining artifacts occupy roughly 3.0 MB across the two methods.

Assets and licenses. The executable benchmark reuses upstream open-source software whose versions and licenses are recorded in the local environment: PyTorch 2.11.0 (BSD-3-Clause), Gymnasium 1.2.3 (MIT), and Minigrid 3.0.0 (MIT). The new assets introduced by this work are the benchmark environments, schema exports, saved result bundle, plotting and table-generation scripts, and the NeurIPS paper package. Those assets are documented by the repository `README.md`, `neurips_latex/BUILD.md`, schema definitions, and the generated result inventory under `results/`.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction explicitly frame the paper as a compact benchmark and evaluation-methodology contribution, not as a new RL optimizer; see the abstract and Sections 1 and 6.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 5 discusses symbolic scope, narrow monitor evidence, the small seed count, and the intentionally limited policy family; Section 6 discusses broader-impact risks from over-interpretation.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present new theorems or formal proofs; its contribution is empirical and evaluative.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections 2–4 define the task tuple, metrics, leakage thresholds, and evaluation protocol, while Appendix A gives the execution context, exact commands, and resource summary used to regenerate the paper artifacts.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The submission supplement is intended to include an anonymized executable artifact with the benchmark code, saved outputs, and the exact regeneration commands listed in Appendix A.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.

- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 2 and 3 specify the task tuple, policy family, step budgets, seed counts, deterministic versus stochastic evaluation, monitor dataset construction, and run-aware monitor splitting; Appendix A adds the exact command path and software context.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Appendix A reports seed variability as mean +/- standard deviation over the three seeds for the core ID proxy/oracle quantities, and the text explicitly avoids stronger inferential claims than that summary supports.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

Justification: Section 3 states the local execution setting, and Appendix A reports the exact Python/runtime context, machine type, wall-clock times, and per-environment retraining budget used in the second-pass rerun.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work involves locally generated symbolic environments, no human subjects, no scraped datasets, and an anonymized submission path; the paper emphasizes conservative claims and explicit reporting of limitations.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 discusses the intended positive impact on evaluation discipline and the main negative risk of over-interpreting or misusing symbolic exploit benchmarks.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.

- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released assets are small symbolic environments and analysis code rather than high-risk generative models, scraped datasets, or dual-use pretrained systems.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper cites the external scientific sources it builds on, and Appendix A records the versions and licenses of the external software assets used in the executable benchmark (PyTorch, Gymnasium, and Minigrid).

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Appendix A identifies the new benchmark assets and points to the repository documentation (`README.md`, `neurips_latex/BUILD.md`, schema definitions, and the generated result inventory) that accompanies them.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The benchmark, environments, training pipeline, and analysis do not use LLMs as part of the core scientific method.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.